

Lab 04

→ Objectives

The objective of this lab is to introduce you to UNIX file I/O functions (open, read, write, close, and lseek) and the make utility.

→ Exercise

In this lab you will learn some of the UNIX file I/O functionality such as:

- open
- read
- write
- seek
- close

We will use an example to learn how to use some of these file I/O functions.

Example #1:

We wrote a C program to copy one file and copy the contents of that file to a new file by performing following steps (here we must provide the input and output file names as *command line arguments*.)

☒ Check if the correct numbers of the argument are given. Three arguments:

- 1) name of the program
 - 2) input file name
 - 3) output file name
- If the number of arguments is not three then the program should print an error message and terminate.
 - What happens if you provide the same file name for input and output?
If you pass the same file name as both the input and output file arguments, the file will be wiped out (truncated to 0 bytes) before you can read from it, causing you to lose all your data.

☒ Use the *open* function in *read only mode* to read the input file.

- The open function takes the name of the file as the first argument and the open flag as the second argument.

- The open flag specifies if the file should be opened in read only mode (**O_RDONLY**), write only mode (**O_WRONLY**), or read-write mode (**O_RDWR**).
- There is an optional third argument that specifies the file permissions called *mode*, we will not use the optional third argument here.
- The third argument specifies the file permissions of the new file created for writing.
- NOTE: UNIX file uses {*read*, *write*, *execute*} (*rw**x*) permissions for the user, group, and everyone.
- You can find out various modes supported by the open function from the “man” page.

☒ The function returns a file descriptor ...

- Typically, a non-negative integer.
- If there is an error opening the file, the function returns -1.
- NOTE: most programs have access to three standard file descriptors: standard input (stdin) – 0, standard output (stdout) – 1, and standard error (stderr) – 2.
- Instead of using the values 0, 1, and 2, we can also use the POSIX name STDIN_FILENO, STDOUT_FILENO, and STDERR_FILENO.

☒ Use the open function ...

- To create a new write for writing the output such that if the file does not exist it will create a new file and if a file with the given name exists it will overwrite the existing file.
- This is accomplished by ORing the different open flags: O_CREAT, O_WRONLY, and O_TRUNC.
- O_CREAT specifies to open a new empty file if the file does not exist and requires the third file permission argument to be provided.
- O_WRONLY specifies that the file should be open for writing only.
- O_TRUNC flag specifies that if the file already exists, then it must be truncated to zero length (destroy any previous data).

☒ Read the file ...

- Read the file by reading fixed chunks of data using the *read* function by providing the file descriptor, input buffer address, and the maximum size of the buffer provided.
- A successful read returns the number of bytes read, 0 if the end-of-file is reached, or -1 if there is an error.
- After you read the data into the buffer, write the buffer to the new file using the *write* function. The *write* function takes the file descriptor, buffer to write, and the number of bytes to write.

- If the write is successful, it will return the number of bytes written.

Example #2

The second example was the *lseek*. We used it to move to location in the file and modify it by performing following steps ...

- ☒ Open the output file of Example #1
- ☒ Use *lseek* to read last 10 bytes of file and print it on console.
 - The *lseek* function takes three arguments: file descriptor, the file offset, and the base address from which the offset is to be implemented (often referred to as *whence*).
 - In this example, we are trying to read the last 10 bytes in the file, so we set the whence to the end of the file using `SEEK_END` and specify the offset as 10.
 - You can look at the man page for *lseek* to find out other predefined whence values: `SEEK_SET`, `SEEK_CUR`, etc.
 - The *lseek* function returns the new file offset on a successful and returns -1 when there is an error
- ☒ Use *lseek* to write a string character "THIS IS NEW MSG FROM LSEEK!"
 - Write at beginning of file.

Sample offset and whence values →

offset = 0, whence = `SEEK_SET`

offset = 0, whence = `SEEK_END`

offset = -1, whence = `SEEK_END`

offset = -10, whence = `SEEK_CUR`

→ Make Utility

Make is a utility that is used to automatically detect which program need to be recompiled while working on a large number of source programs and will recompile only those programs that have been modified. The *make* utility uses a *Makefile* to describe the rules for determining the dependencies between the various programs and the compiler and compiler options to use for compiling the programs. In case of C programs, an executable is created from object files (*.o files) and object files are created from source files. Source files are often divided into header files (*.h files) and actual source files (*.c files).

To illustrate the use of *make*, let us consider adding a new function to measure the time taken by the insertion sort program that we wrote in Lab 2. Instead of adding this method to the same file

as the insertion sort, let us create a new file and create a header file that has the method prototype.

☒ `gettime.h`

☒ `gettime.c`

NOTE: we can compile the file `gettime.c` separately and link the object file with any other program that uses the `gettime` function. To use the `gettime` function in the insertion sort program, we have to include the file `gettime.h` and invoke the `gettime` function before and after the call to `insertionsort` function.

Terminal →

```
$ gcc -c gettime.c
```

```
$ gcc -c insertionsort.c
```

```
$ gcc -o insertionsort insertionsort.o gettime.o
```

NOTE: we don't have to recompile `gettime.c` if we are only making changes to the file `insertionsort.c`. These dependencies is what we can describe in a make file and let the make utility determine which files need to be updated and recompile those files.

Key Elements in a Makefile:

- 1) Any text after `#` is a comment and is ignored by the make utility
- 2) Constants (they are referred to as macros) are assigned values using the notation:
constant = values1 value2
- 3) You use these constants in rules we have to use `$(NAME)`, where `NAME` is a constant defined earlier
- 4) Dependencies are denoted using the notation: *leftside:<tab>rightside* and is followed by what commands to execute in the next line after a tab (it is important that call commands start with a tab, otherwise make will complain about missing separator)
- 5) The symbol `%` is used to denote wildcard entries, e.g., `%.o:%.c` implies that all `.o` files depend on corresponding `.c` files
- 6) Special macro `$$` represents text to the left of `:` and `$$^` represents text to the right of `:` whereas `$$<` represents the first item in text that appears to the right of `:`

If the Makefile is saved as `Makefile` or `makefile`, you can invoke make utility by typing `make`. If you use a different file name other than `Makefile` or `makefile` then you have to specify the `makefile` using the `-f` option to `make`.

If you type `make` →

\$ make

gcc -Wall -g -c -o gettime.o gettime.c

gcc -Wall -g -c -o insertionsort.o insertionsort.c

gcc -Wall -g -o insertionsort gettime.o insertionsort.o

If you change gettime.h then you should see all files recompiled and the following output:

\$ touch gettime.h

\$ make

gcc -Wall -g -c -o gettime.o gettime.c

gcc -Wall -g -c -o insertionsort.o insertionsort.c

gcc -Wall -g -o insertion gettime.o insertionsort.o

If you change gettime.c then you should see the following output:

\$ touch gettime.c

\$ make

gcc -Wall -g -c -o gettime.o gettime.c

gcc -Wall -g -o insertionsort gettime.o insertionsort.o

If you only change insertionsort.c you will see the following output:

\$ touch insertionsort.c

\$ make

gcc -Wall -g -c -o insertionsort.o insertionsort.c

gcc -Wall -g -o insertionsort gettime.o insertionsort.o

If you have not modified any files, if you execute make, you will see that following output:

\$ make

Make: Nothing to be done for 'all'.

→ Homework

Implement a C program:

- ☐ takes two filename as command-line arguments
- ☐ concatenates the contents of the second file to the first file
- ☐ check if the two filenames provided are the same and print an error message if so
- ☐ first file should contain it's original content and the contents of the second file

- ☒ second file should be unchanged
- ☒ create a file called REAMME.txt
- ☒ include instructions for compiling and executing your program in README.txt